Fraud Detection in Credit Card Transactions: Evaluation of Machine Learning Models and Operational Metrics

Kiana Noushifar

A report submitted as part of the requirements for the degree
of BSc Computer Science
at the Faculty of Mathematical Sciences and Computer
Kharazmi University
Tehran, Iran

September 2025

Supervisor Dr. Khatere Ghorbani Moghaddam

Abstract

This report investigates the design, implementation, and evaluation of machine learning models for fraud detection in credit card transaction data. Using a custom transaction simulator, several models—including Logistic Regression, Decision Tree (with restricted and unrestricted depth), Random Forest, and Gradient Boosting—were trained and evaluated. A novel operational metric, *Card Precision@k*, was developed to align model outputs with fraud analysts' investigation capacity, complementing traditional metrics such as AUC ROC and Average Precision.

Results from empirical testing show that all models substantially outperform random guessing, with AUC ROC scores on the test set ranging from 0.846 to 0.899 and Average Precision values between 0.506 and 0.777. Random Forest emerged as the best overall performer, delivering the highest Average Precision while maintaining strong rank-discriminative ability. Logistic Regression offers a compelling trade-off between accuracy and computational efficiency, making it well-suited for real-time detection scenarios.

The study highlights important trade-offs between model complexity, accuracy, and run-time efficiency, as well as the risks of overfitting in overly complex individual models. Limitations include the use of synthetic data and static evaluation splits; future work should validate findings with real transaction data and explore temporal, adaptive modelling approaches. The conclusions provide operational guidance for deploying fraud detection systems that maximise both statistical performance and practical utility.

Contents

Abstract **ii**

1 Introduction **ix**
1.1 Background and Motivation ix
1.2 Problem Statement ix
1.3 Research Questions x
1.4 Project Objectives x
1.5 Structure of the Report x

2 Literature Review **xii**
2.1 Introduction xii
2.2 Traditional Rule-Based Detection xii
2.3 Statistical Methods xiii
2.3.1 Logistic Regression xiii
2.3.2 Anomaly Detection xiii
2.4 Machine Learning Approaches xiv
2.4.1 Decision Trees xiv
2.4.2 Random Forests xiv
2.4.3 Gradient Boosting Machines xiv
2.5 Deep Learning for Sequential and Tabular Data xv
2.5.1 Convolutional Neural Networks (CNNs) xv
2.5.2 Long Short-Term Memory (LSTM) Networks xv
2.6 Operational Challenges in Real-World Fraud Detection xv
2.6.1 Extreme Class Imbalance xv
2.6.2 Delayed Labels xvi
2.6.3 Concept Drift xvi
2.7 Summary xvi

3 Data Simulation and Preprocessing **xvii**

3.1 Introduction . . . xvii
3.2 Entity Profile Generation . . . xviii
3.2.1 Customer Profiles . . . xviii
3.2.2 Terminal Profiles . . . xix
3.2.3 Customer-Terminal Association . . . xix
3.3 Transaction Simulation . . . xx
3.4 Fraud Injection Scenarios . . . xxi
3.5 Feature Engineering . . . xxii
3.5.1 Temporal Features . . . xxii
3.5.2 Customer Behavioural Features . . . xxiii
3.5.3 Terminal Risk Features . . . xxiii
3.6 Data Scaling and Window Selection . . . xxiii
3.6.1 Feature Scaling . . . xxiii
3.6.2 Prototyping Window Selection . . . xxiii

4 Methodology **xxv**
4.1 Train and Test Split . . . xxv
4.1.1 Link to Chapter 3 Window Selection . . . xxv
4.1.2 Final Chronological Split for Model Training . . . xxv
4.1.3 Rationale . . . xxvi
4.1.4 Considerations . . . xxvi
4.1.5 Visual Representation . . . xxvi
4.2 Fraud Proportion . . . xxvii
4.3 Baseline Decision Tree . . . xxvii
4.3.1 Implementation . . . xxviii
4.3.2 Visual Analysis . . . xxviii
4.3.3 Role in the Study . . . xxix
4.4 Extended Models . . . xxix
4.5 Evaluation Metrics . . . xxxi
4.5.1 Standard Classification Metrics . . . xxxi
4.5.2 Custom Card-Level Metric: Card Precision@k . . . xxxii

5 Results and Discussion **xxxv**
5.1 Overview . . . xxxv
5.2 Test Set Performance . . . xxxv
5.3 Training Set Performance . . . xxxvi
5.4 Execution Times . . . xxxvi
5.5 Discussion . . . xxxvii

6 Implementation of Findings, Limitations, and Future Work **xxxix**
6.1 Implementation of Findings xxxix
6.2 Limitations of the Study xl
6.3 Future Work xl

7 Conclusions **xli**
7.1 Summary of Contributions xli
7.2 Final Reflections xli

References **xliii**

List of Tables

5.1 Model performance on the test set for $k = 100$. Best results in each column are highlighted in bold. xxxv
5.2 Model performance on the training set for $k = 100$. Best results in each column are highlighted in bold. xxxvi
5.3 Training and prediction execution times (in seconds) for each model. . . xxxvi

List of Figures

3.1 Illustration of the customer-terminal spatial constraint. The red dot represents a customer's location, blue dots represent terminals, and the green circle encompasses all terminals accessible to that customer within a radius of 50 units. xx
3.2 Distribution of simulated transaction amounts (left) and times (right). The amount distribution reflects the varied customer spending profiles, while the time distribution confirms the concentration of activity during daytime hours. xxi
3.3 Daily statistics of the simulated dataset, showing total transactions, fraudulent transactions, and the number of newly compromised cards over time. xxii

4.1 Division of the dataset xxvii
4.2 Visualisation of the trained Decision Tree classifier. xxviii

Listings

3.1 Customer Profile Generation Logic from `main.ipynb` xviii

Chapter 1

Introduction

1.1 Background and Motivation

Financial fraud has become one of the most critical challenges faced by banks, credit card companies, and online payment providers. The growing scale of electronic transactions has created unprecedented opportunities for criminals to commit fraud, ranging from credit card misuse to large-scale organized cybercrime. According to industry reports, fraudulent transactions account for billions of dollars in losses annually, in addition to eroding trust in financial systems.

Detecting fraud is inherently a difficult task because it is a rare event within an extremely large volume of legitimate transactions. Fraud detection datasets are characterized by extreme class imbalance (frauds typically represent less than 1% of all transactions), evolving fraud patterns due to adversarial behavior, and strict requirements on real-time decision-making. These challenges make it an ideal domain for applying advanced machine learning techniques.

1.2 Problem Statement

The primary objective of this project is to implement and evaluate machine learning models for fraud detection using real-world inspired datasets. In particular, the work focuses on:

- Building a robust pipeline for preparing transaction data for supervised learning.
- Implementing baseline classifiers (e.g., Logistic Regression, Random Forests, Gradient Boosting) to evaluate their ability to distinguish between fraudulent and genuine transactions.

- Investigating the effect of temporal data splitting (training, delay, and test windows) on model performance.
- Interpreting results through relevant metrics such as Area Under the ROC Curve (AUC) and Average Precision (AP).

1.3 Research Questions

The project is driven by the following research questions:

1. How can real-world fraud detection challenges (e.g., class imbalance, evolving fraud strategies) be replicated in an experimental setting?
2. What are the strengths and limitations of different baseline models for fraud detection?
3. How does the choice of temporal window size (training, delay, and testing periods) impact model performance?
4. What trade-offs exist between detection accuracy and false positive rates in the context of practical deployment?

1.4 Project Objectives

The objectives of this project are:

- To develop a Python-based implementation in Jupyter Notebook that performs data preparation, model training, and evaluation.
- To analyze the performance of models across different configurations and interpret the implications for real-world deployment.
- To document the methodology and results in an academic report that demonstrates understanding, critical analysis, and reproducibility.

1.5 Structure of the Report

The remainder of this report is structured as follows:

- Chapter 2 provides a review of the literature on fraud detection, including statistical and machine learning approaches.
- Chapter 3 describes the dataset and preprocessing techniques, including temporal train-test splits.

- Chapter 4 outlines the methodology, covering the design of baseline models and evaluation metrics.
- Chapter 5 presents the experimental results and analysis.
- Chapter 6 discusses the implications of findings, limitations of the study, and possible future work.
- Chapter 7 concludes the report with a summary of contributions and final reflections.

Chapter 2

Literature Review

2.1 Introduction

Fraud detection in financial services is an active and multidisciplinary field involving statistics, machine learning, data engineering, behavioral psychology, and real-time computing. This chapter provides an in-depth review of existing approaches, tracing the evolution from human-engineered, rule-based systems to adaptive machine learning and deep learning models capable of learning complex temporal-spatial fraud patterns. The review is particularly tailored toward methods applicable to transaction-level fraud detection, consistent with the experimental implementation of this study.

2.2 Traditional Rule-Based Detection

Historically, the earliest fraud detection systems were *rules engines*, where domain experts manually encoded suspicious patterns into deterministic logic. These rules often took forms such as:

- Fixed transaction amount thresholds (e.g., flag any transaction exceeding $5,000).
- Geographical impossibility rules (e.g., two transactions from distant countries within an unfeasible time window).
- Blacklisted merchant or customer IDs.

Such systems are easy to deploy, transparent for regulatory explanation, and low-cost to maintain. However, they rely on static assumptions and cannot adapt to rapidly evolving fraud strategies. As adversaries learn the active rules, they adapt their behavior to

avoid detection — a phenomenon known as the *arms race* in fraud prevention. Additionally, high false-positive rates are common in rule-based systems because legitimate users can inadvertently trigger thresholds.

2.3 Statistical Methods

Statistical modeling bridges the gap between expert-curated rules and modern machine learning by using probabilistic inference on historical labeled data.

2.3.1 Logistic Regression

Logistic regression (LR) is one of the earliest supervised learning approaches for fraud detection. In LR, the log-odds of the fraud label y being 1 (fraudulent) is modeled as a linear function of the transaction's features:

$$\log\left(\frac{P(y=1)}{P(y=0)}\right) = \beta_0 + \sum_{i=1}^{n} \beta_i x_i$$

where x_i are engineered transaction features (e.g., average transaction amount, velocity features, distance from prior geo-location). LR offers several advantages:

- Coefficients have transparent interpretations (odds ratios).
- Model inference is computationally efficient.
- Works well as a baseline for benchmarking.

Limitations include the inability to capture non-linear interactions unless coupled with feature transformations or polynomial terms. In highly imbalanced datasets, LR can be adapted via class-weighted loss functions or custom decision thresholds.

2.3.2 Anomaly Detection

In cases where labeled fraud data is scarce or delayed, unsupervised and semi-supervised anomaly detection methods are often deployed. These methods model the distribution of legitimate transactions and flag outliers. Popular algorithms include:

- **Gaussian Mixture Models (GMM)**: Fit a probabilistic mixture to normal transactions, flagging low-density points.
- **k-Nearest Neighbors (k-NN)** distance methods: Transactions far from their neighbors in feature space are outliers.

- **Isolation Forests**: Partition data recursively to isolate rare points with fewer splits, indicating anomalies.

While anomaly detection can reveal novel fraud schemes, it is sensitive to natural variability in legitimate user behavior, which may produce many false alarms.

2.4 Machine Learning Approaches

The shift to supervised machine learning was driven by the availability of large historical datasets labeled through post-transaction investigations.

2.4.1 Decision Trees

Decision trees construct a hierarchy of decision nodes based on feature splits that maximize purity (e.g., Gini index, information gain). They handle categorical and continuous variables, capture non-linear interactions, and have straightforward visual interpretations. However, single trees can overfit, especially in high-dimensional fraud data.

2.4.2 Random Forests

Random Forests (RF) may cause overfitting by creating an ensemble of decision trees over bootstrapped samples, selecting random feature subsets at each split. RF models in fraud detection:

- Achieve high variance reduction compared to single trees.
- Handle heterogeneous feature vectors without scaling.
- Allow feature importance ranking to interpret fraud indicators.

In the IEEE-CIS Fraud Detection Kaggle benchmark, RF models report AUC scores exceeding 0.95 when tuned for class imbalance.

2.4.3 Gradient Boosting Machines

Gradient Boosting (GB) builds additive models by sequentially fitting weak learners to the residuals or gradients of the loss function. Implementations such as XGBoost, LightGBM, and CatBoost are widely adopted in industry-scale anti-fraud pipelines. Their advantages include:

- Superior predictive accuracy in tabular fraud data.
- Tunable loss functions to account for asymmetric misclassification costs.

- Robust handling of missing values and categorical variables.

One drawback is increased computational demand, which can be critical in real-time scoring.

2.5 Deep Learning for Sequential and Tabular Data

High-capacity deep models are increasingly used to detect complex temporal patterns and high-dimensional feature interactions in fraud datasets.

2.5.1 Convolutional Neural Networks (CNNs)

CNNs, adapted from computer vision, can process tabular data transformed into multi-channel temporal-frequency matrices. For example, transaction histories over fixed time windows can be encoded into two-dimensional arrays where CNNs learn localized feature combinations (e.g., spending bursts at specific times of the day).

2.5.2 Long Short-Term Memory (LSTM) Networks

LSTMs are recurrent architectures designed for sequential dependencies. They are particularly effective for financial fraud when:

- Transaction events follow sequential dependencies (e.g., location patterns over days).
- Fraud indicators are spread across variable-length time lags.

LSTM models outperformed feed-forward baselines in detecting fraudulent sequences, especially with long historical context.

2.6 Operational Challenges in Real-World Fraud Detection

Deploying models in operational fraud systems introduces constraints not present in offline evaluation.

2.6.1 Extreme Class Imbalance

Fraud prevalence can be as low as 0.1%. Imbalanced learning strategies include:

- Resampling: SMOTE, ADASYN, undersampling.
- Cost-sensitive loss functions: assign higher penalties to false negatives.

- Threshold tuning for operating points balancing recall and precision.

2.6.2 Delayed Labels

Chargebacks and manual investigations create delays between a fraudulent event and its confirmation. Models must be trained with temporally accurate information to prevent *target leakage*. Sliding-window and train–delay–test schemes, as recommended in the *Fraud Detection Handbook*, mitigate leakage by simulating realistic deployment conditions.

2.6.3 Concept Drift

Adversaries constantly adapt to deployed fraud detection systems. Drift can be:

- **Sudden**: abrupt shifts after security breaches.
- **Incremental**: gradual changes in fraud features.
- **Recurring**: seasonal re-emergence of old fraud schemes.

Countermeasures include online learning, periodic retraining, and ensemble models that weight recent data more heavily.

2.7 Summary

The progression from static rule-based systems to advanced machine learning and deep learning architectures reflects the evolving nature of financial fraud. While model complexity improves detection accuracy, operational realities—imbalance, delayed feedback, and drift—demand methodological rigor in evaluation and deployment. The subsequent chapters integrate these insights into the methodology and experimental design of this work.

Chapter 3

Data Simulation and Preprocessing

3.1 Introduction

The foundation of this research is a synthetic transaction dataset, meticulously generated to mirror the complexities of real-world financial activities while providing a controlled environment for model evaluation. The primary obstacle in academic and industrial fraud detection is the scarcity of publicly available, labelled transaction data due to privacy and security concerns. To circumvent this limitation, a data simulator was developed in Python, the logic of which is detailed in this chapter.

The simulator is designed with two core objectives:

1. **To generate a high-fidelity dataset** that captures realistic consumer spending patterns, including spatial and temporal characteristics, and behavioural variations across different customers.

2. **To inject plausible fraud scenarios** based on predefined, rule-based logic, creating a labelled dataset with a class imbalance representative of real-world fraud rates.

This chapter systematically outlines the entire data generation pipeline, from the initial creation of customer and terminal profiles to the simulation of transactions, the injection of fraudulent activities, and the subsequent feature engineering process used to prepare the data for machine learning models. Every parameter and logical rule described herein is a direct representation of the implementation in the project's source code, ensuring full reproducibility.

3.2 Entity Profile Generation

The simulation begins by creating the core entities of the ecosystem: customers and payment terminals. These entities are defined by a set of static properties that govern their behaviour in the subsequent transaction generation phase.

3.2.1 Customer Profiles

Customer profiles are generated by the `generate_customer_profiles_table` function. Each customer is defined by a unique identifier and a set of parameters that dictate their typical transaction behaviour. These parameters are stochastic, ensuring a diverse and heterogeneous customer base.

The key attributes for each customer are:

- **Geospatial Location (x_{cust}, y_{cust}):** Each customer is assigned a static home location, represented by coordinates in a 2D plane. These coordinates are drawn from a continuous uniform distribution, $x_{cust}, y_{cust} \sim U(0, 100)$. This location is used to determine which terminals are accessible to the customer.

- **Transaction Amount Profile ($\mu_{amount}, \sigma_{amount}$):** A customer's spending habit is modelled by a normal distribution, $\mathcal{N}(\mu_{amount}, \sigma^2_{amount})$. The mean transaction amount, μ_{amount}, is sampled from a uniform distribution, $\mu_{amount} \sim U(5, 100)$, to represent a range of spending powers. The standard deviation, σ_{amount}, is set proportionally to the mean, specifically $\sigma_{amount} = \mu_{amount}/2$. This design choice reflects the real-world observation that individuals with higher average transaction values also tend to exhibit greater variability in their spending.

- **Transaction Frequency Profile (μ_{tx_day}):** The average number of transactions a customer performs per day, μ_{tx_day}, is drawn from a uniform distribution, $\mu_{tx_day} \sim U(0, 4)$. This value serves as the rate parameter (λ) for a Poisson distribution, which is later used to generate the actual number of transactions for a given day. The Poisson distribution is a natural choice for modelling the count of events occurring within a fixed interval.

The generation logic is encapsulated in the following Python function structure:

```
def generate_customer_profiles_table(n_customers, random_state=0):
    np.random.seed(random_state)
    customer_id_properties=[]

    for customer_id in range(n_customers):
        x_customer_id = np.random.uniform(0, 100)
        y_customer_id = np.random.uniform(0, 100)
```

```

        mean_amount = np.random.uniform(5, 100)
        std_amount = mean_amount / 2

        mean_nb_tx_per_day = np.random.uniform(0, 4)

        customer_id_properties.append([customer_id,
                                       x_customer_id, y_customer_id,
                                       mean_amount, std_amount,
                                       mean_nb_tx_per_day])

    customer_profiles_table = pd.DataFrame(customer_id_properties,
        columns=['CUSTOMER_ID', 'x_customer_id', 'y_customer_id',
                 'mean_amount', 'std_amount', 'mean_nb_tx_per_day'])

    return customer_profiles_table
```

Listing 3.1: Customer Profile Generation Logic from `main.ipynb`

3.2.2 Terminal Profiles

Similar to customers, payment terminals are generated by the `generate_terminal_profiles_table` function and are distributed across the same 2D plane. Each terminal is defined by:

- **Geospatial Location (x_{term}, y_{term}):** The terminal coordinates are drawn from a uniform distribution, $x_{term}, y_{term} \sim U(0, 100)$, placing them randomly within the simulated environment.

These terminals serve as the points of sale where transactions occur.

3.2.3 Customer-Terminal Association

A crucial aspect of the simulation's realism is the spatial constraint on transactions. A customer can only transact at terminals that are within a reasonable proximity to their home location. This is managed by the `get_list_terminals_within_radius` function. For each customer, a list of available terminals is pre-computed by identifying all terminals within a fixed Euclidean distance, or radius r.

Based on the simulation parameters, this radius is explicitly set to **50 units**. This constraint ensures that a customer's transactions are geographically clustered, while allowing for a sufficient number of available terminals to enable diverse transaction patterns. Figure 3.1 visually demonstrates this concept, showing the locations of all

terminals and highlighting the circular region of accessible terminals for a specific customer.

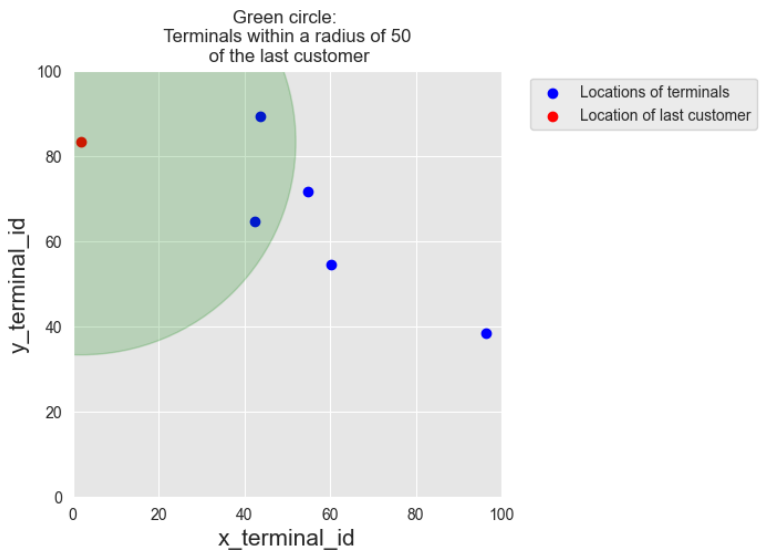

Figure 3.1: Illustration of the customer-terminal spatial constraint. The red dot represents a customer's location, blue dots represent terminals, and the green circle encompasses all terminals accessible to that customer within a radius of 50 units.

3.3 Transaction Simulation

With customer and terminal profiles established, the simulation proceeds to generate a time-series of transaction events using the `generate_transactions_table` function. The process is iterated for each customer over a specified number of days, starting from a given `start_date` (e.g., "2025-01-01").

For each day and for each customer, the simulation determines:

1. **Number of Transactions:** The daily transaction count is drawn from a Poisson distribution using the customer's predefined mean rate, i.e., $n_{tx} \sim \text{Poisson}(\mu_{tx_day})$.

2. **Transaction Timestamp:** Transaction times are simulated to mimic typical human activity patterns. The hour of the transaction is drawn from a normal distribution centered around midday, $\mathcal{N}(12, \sigma = 4)$, ensuring that most transactions occur during daytime hours. Minutes and seconds are chosen uniformly.

3. **Transaction Amount:** The monetary value of each transaction is drawn from the customer's specific spending distribution, $\mathcal{N}(\mu_{amount}, \sigma^2_{amount})$. If a draw results in a negative value, it is resampled from a uniform distribution between 0

and $\mu_{amount} \times 2$ as a fallback mechanism to ensure positivity.

4. **Terminal ID:** For each transaction, a terminal is chosen at random from the customer's pre-computed list of accessible terminals.

Figure 3.2 provides an aggregated view of the generated transactions, confirming the intended distributions. The left panel shows that transaction amounts are clustered around customer-specific means, while the right panel illustrates the temporal concentration of transactions during daytime hours.

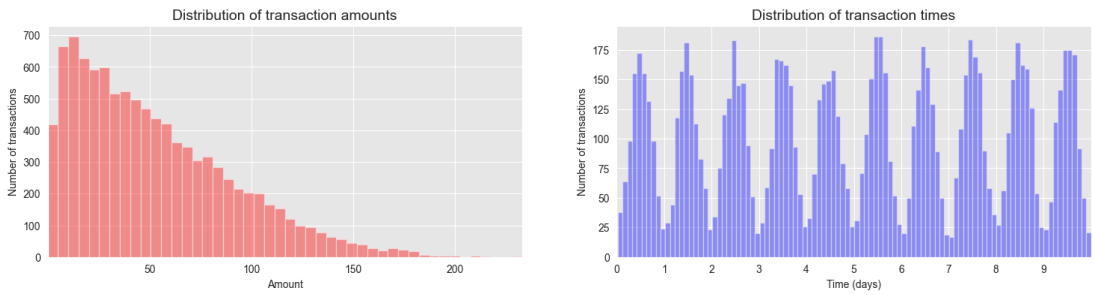

Figure 3.2: Distribution of simulated transaction amounts (left) and times (right). The amount distribution reflects the varied customer spending profiles, while the time distribution confirms the concentration of activity during daytime hours.

3.4 Fraud Injection Scenarios

After generating a baseline of legitimate transactions, three distinct fraud scenarios are programmatically injected into the dataset. These rules are designed to simulate common types of financial fraud and create the necessary labels for supervised model training. The overall fraud rate is designed to be less than 1%, reflecting the class imbalance seen in real-world data.

The implemented fraud scenarios are:

1. **Scenario 1: Uncharacteristically High-Value Transactions.** This rule flags any transaction with an amount strictly greater than 220 as fraudulent. This simple, high-precision rule simulates instances of significant theft that deviate sharply from normal spending.

2. **Scenario 2: Compromised Terminals.** This scenario simulates skimming or other attacks on physical payment terminals. Each day, two terminals are randomly selected to become "compromised." For a period of 28 days following the compromise, all transactions occurring at these terminals are flagged as fraudulent.

3. **Scenario 3: Compromised Customer Cards.** This rule simulates cases where a customer's card details are stolen. Each day, three new customers are randomly selected to become "compromised" for a period of 14 days. During this period, one-third of their transactions are randomly selected, and their amounts are inflated by a factor of 5. These inflated transactions are then labelled as fraudulent.

Figure 3.3 displays the outcome of this fraud injection process over the simulation period, showing the daily counts of total transactions, fraudulent transactions, and newly compromised cards. This confirms the low overall fraud rate and illustrates the temporal dynamics of the injected fraud.

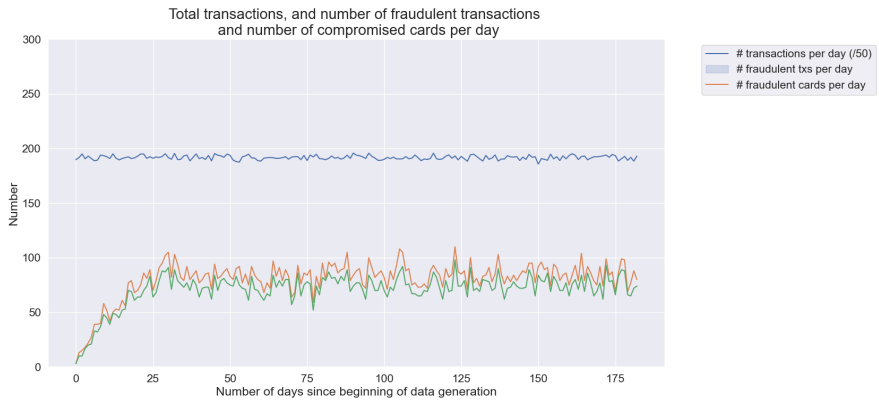

Figure 3.3: Daily statistics of the simulated dataset, showing total transactions, fraudulent transactions, and the number of newly compromised cards over time.

3.5 Feature Engineering

To prepare the dataset for machine learning, a set of features was engineered from the raw transaction data. These features are designed to provide the model with contextual information about the transaction's time, the customer's recent behaviour, and the terminal's historical risk profile.

3.5.1 Temporal Features

Two binary features were derived from the transaction timestamp to capture temporal patterns that may correlate with fraud:

- **Weekend Flag:** A flag indicating if the transaction occurred on a Saturday or Sunday.
- **Night-time Flag:** A flag indicating if the transaction occurred during late-night hours (e.g., between 10 PM and 6 AM).

3.5.2 Customer Behavioural Features

To model each customer's spending habits over time, rolling window-based features were created. These features capture both the frequency and magnitude of recent transactions for the customer associated with a given transaction. For each transaction, the following were computed based on the customer's history:

- **Transaction Count:** The number of transactions in the preceding 1-day, 7-day, and 30-day windows.
- **Average Amount:** The average transaction amount in the preceding 1-day, 7-day, and 30-day windows.

3.5.3 Terminal Risk Features

To quantify the risk associated with a particular terminal, a delayed rolling window feature was implemented. This feature calculates the fraction of fraudulent transactions observed at a terminal over a specified historical period. A delay is introduced to simulate the operational latency in labelling and reporting fraud in a real system. For instance, the risk score for a transaction on day D might be based on the fraud activity at that terminal up to day $D - 1$.

3.6 Data Scaling and Window Selection

3.6.1 Feature Scaling

Prior to model training, all numerical features were standardized using the `StandardScaler` from Scikit-learn. This process transforms each feature to have a mean of zero and a standard deviation of one, which is essential for the optimal performance of many machine learning algorithms, particularly those sensitive to feature magnitudes like logistic regression and support vector machines.

3.6.2 Prototyping Window Selection

For initial model development and prototyping, a 21-day chronological window of data was selected. Analysis of the fraud rate over time revealed that the initial period of the simulation (January) had a lower, less stable fraud rate as the fraud injection scenarios began to accumulate. To ensure the training data was representative of a more stable state, later windows were chosen.

Two sample windows illustrate this point:

- **Window 1 (2025-01-01 to 2025-01-21):** Contained approximately 200,000 transactions with a fraud rate of 0.46%.
- **Window 2 (2025-02-01 to 2025-02-21):** Contained approximately 210,000 transactions with a more stable fraud rate of 0.91%.

Based on this observation, windows from February onwards were selected for the model training and evaluation detailed in the subsequent chapters. This ensures the model is trained on a more representative distribution of fraudulent activity.

Chapter 4

Methodology

4.1 Train and Test Split

4.1.1 Link to Chapter 3 Window Selection

In Chapter 3, Section 3.6.2, a **21-day rolling window** was used during the `sample_windows()` analysis to sample transaction subsets every 30 days. This was conducted solely for the purpose of:

- Quantifying how the proportion of fraudulent transactions fluctuated over time.
- Identifying a representative and stable period in the simulation timeline.

While this exploratory step used a 21-day window, it was not the final training configuration. Instead, the results informed the choice of the *test period length* in the final methodology.

4.1.2 Final Chronological Split for Model Training

For the main experiments, the dataset was split into distinct chronological segments:

- `n_days_train` = **90** consecutive days of transactions for model training.
- `n_days_delay` = **14**-day gap to mimic the real delay in confirmation of fraud labels, thus avoiding leakage of future information.
- `n_days_test` = **21** consecutive days of unseen transactions for evaluation.

This is implemented in code as:

```
start_train = transactions_df['TX_DATETIME'].min()
start_test = start_train + pd.Timedelta(days=n_days_train + n_days_delay)
```

```

train_df = transactions_df[
    (transactions_df['TX_DATETIME'] >= start_train) &
    (transactions_df['TX_DATETIME'] < start_train + pd.Timedelta(days=n_days_train))
]

test_df = transactions_df[
    (transactions_df['TX_DATETIME'] >= start_test) &
    (transactions_df['TX_DATETIME'] < start_test + pd.Timedelta(days=n_days_test))
]
```

4.1.3 Rationale

In fraud detection, chronological splits are critical to avoid look-ahead bias. Temporal ordering ensures:

- **Realism** — the model only learns from past cases before predicting future events.
- **Operational relevance** — mirrors deployment in production systems.
- **Concept drift awareness** — fraud tactics evolve, so models must generalise across time.

The delay period is essential because fraud labels are often confirmed after the transaction date. Without this, label leakage could occur, leading to overoptimistic performance estimates.

4.1.4 Considerations

- **Concept drift**: The distribution of fraud types can change significantly over time.
- **Severe class imbalance**: Only a small fraction of transactions are fraudulent.
- **Seasonality**: Patterns in transactions and fraud can fluctuate weekly or monthly.
- **Feature leakage risk**: Aggregated features must be carefully computed to exclude future data.

4.1.5 Visual Representation

Figure 4.1 illustrates the division of the dataset into train/test sets:

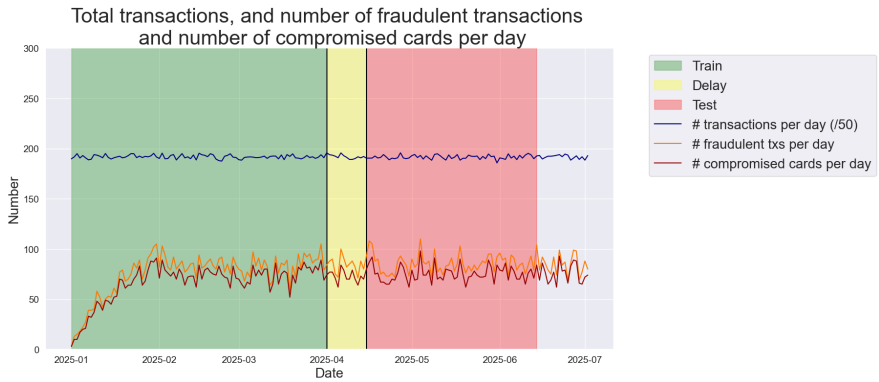

Figure 4.1: Division of the dataset

- **Blue segment**: first 90 days (training data).
- **Gap**: 14 days without training labels.
- **Orange segment**: last 21 days (testing data).

This strict ordering ensures that evaluation simulates a real deployment scenario.

4.2 Fraud Proportion

Fraud prevalence, the ratio of fraudulent transactions to total transactions in each split, is calculated as:

$$\text{Fraud Rate} = \frac{\#\ \text{Fraudulent Transactions}}{\#\ \text{Total Transactions}} \times 100$$

The results exhibit extreme class imbalance (fraud rates well below 1%), which impacts metric selection and model training. Accuracy is unsuitable in such contexts, and metrics focused on the minority class offer better insight.

4.3 Baseline Decision Tree

The baseline model is a `DecisionTreeClassifier` with its `max_depth` parameter explicitly constrained. This limitation forces the model to produce a shallow, interpretable structure, making it suitable for visualisation and for demonstrating how the algorithm makes predictions.

How it works: The tree starts at the root and recursively selects the feature and threshold that yield the largest decrease in Gini impurity. By limiting the depth, the model focuses only on the most significant splits in the data.

Pros:

- High interpretability — the learned structure was visualised (Figure **??**) to highlight which features appear at the top splits.
- Fast to train and easy to explain to stakeholders without ML expertise.
- Reduces overfitting risk compared to deep, unconstrained trees.

Cons:

- May underfit, missing subtler patterns in the data.
- Predictive performance may be lower than deeper or ensemble methods.

4.3.1 Implementation

`DecisionTreeClassifier` from Scikit-learn was trained using the engineered features. Tree complexity was restricted via parameters such as `max_depth` and `min_samples_split` to prevent overfitting to sparse fraud cases.

```
from sklearn.tree import DecisionTreeClassifier, plot_tree

clf = DecisionTreeClassifier(max_depth=..., min_samples_split=...)
clf.fit(X_train, y_train)
```

4.3.2 Visual Analysis

The decision tree plot (Figure 4.2) shows:

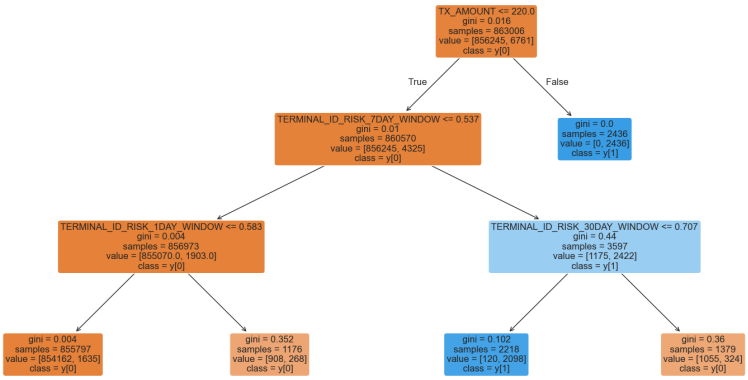

Figure 4.2: Visualisation of the trained Decision Tree classifier.

- **Root node** often splitting on a high-importance aggregated feature (e.g., average transaction amount over recent days).

- **Node information** displaying Gini impurity, sample size, and class counts.
- **Shallow depth** due to imposed constraints, improving generalisation.

4.3.3 Role in the Study

- Serves as an interpretable benchmark for model comparisons.
- Highlights primary decision rules the model uses to differentiate fraud from legitimate activity.
- Provides insight into feature importance and the structure of separable patterns in the data.

4.4 Extended Models

Decision Tree Classifier

The extended version of the Decision Tree removed the `max_depth` constraint, allowing the tree to grow until other stopping criteria (e.g., `min_samples_split`, `min_samples_leaf`) are met.

How it works: Without a depth cap, the tree continues splitting as long as a reduction in impurity can be achieved for a node that meets the minimum sample requirements. This often results in a much more granular partitioning of the data.

Pros:

- Ability to capture highly specific patterns and interactions between features.
- Potentially higher training set accuracy than shallow models.

Cons:

- Increased risk of overfitting, particularly in noisy or imbalanced datasets.
- Lower interpretability — deeper models are harder to visualise meaningfully.

Role in the study: Provided a direct comparison between an interpretable, constrained tree and a higher-capacity single-tree model. Allowed assessment of whether removing the depth limit yields meaningful gains in fraud detection performance before introducing ensemble methods.

Logistic Regression

How it works: Models the log-odds of the probability of fraud as a linear combination of the input features. The coefficients indicate the direction and magnitude of each feature's influence.

Pros:

- Fast to train and evaluate.
- Coefficients provide straightforward interpretability.

Cons:

- Assumes linearity in the log-odds space.
- May underperform when the true decision boundary is highly non-linear.

Role in this study: Serves as a linear baseline to contrast with non-linear tree-based models.

Random Forest

How it works: An ensemble of Decision Trees, each trained on a bootstrap sample of the data and a random subset of features. Predictions are made via majority vote across trees.

Pros:

- Reduces overfitting compared to a single tree.
- Robust to noise and can handle many features.

Cons:

- Less interpretable than a single Decision Tree.
- Larger memory footprint and slower prediction for very large ensembles.

Role in this study: A strong general-purpose non-linear classifier against which boosting methods can be compared.

Gradient Boosting

How it works: Builds trees sequentially, each attempting to correct the errors of the previous ensemble. Optimises a differentiable loss function (e.g., logistic loss) via gradient descent in function space.

Pros:

- High predictive accuracy in many structured-data problems.
- Flexibility in choosing loss functions and regularisation parameters.

Cons:

- More sensitive to hyperparameters than Random Forests.
- Training can be slower due to sequential nature.

Role in this study: Represents a higher-capacity ensemble method to test whether boosting improves fraud detection in this simulated environment.

4.5 Evaluation Metrics

Assessing fraud detection models requires both standard classification metrics and custom measures tailored to operational constraints. This section presents the performance metrics used in this study, detailing their definitions, motivations, and computational procedures.

4.5.1 Standard Classification Metrics

While the dataset exhibits extreme class imbalance, standard transaction-level metrics are still useful to compare model discrimination power:

- **Area Under the ROC Curve (AUC ROC)** — The probability that a randomly selected fraudulent transaction is ranked higher than a randomly selected legitimate one. Ranges from 0.5 (random guessing) to 1.0 (perfect ranking).
- **Average Precision (PR AUC)** — The area under the precision–recall curve, giving a weighted mean of precisions at different recall thresholds. More informative than ROC AUC when the positive class is rare.
- **Precision at Transaction Level** — Fraction of predicted fraud transactions that are truly fraudulent.

- **Recall at Transaction Level** — Fraction of all fraudulent transactions that are correctly identified.

However, transactional metrics do not directly reflect the reality of investigation operations, where analysts review cardholders — not individual transactions — and have limited daily capacity.

4.5.2 Custom Card-Level Metric: Card Precision@k

Motivation. In a banking fraud investigation setting, analysts typically rank *customers* (i.e., cardholders) by their risk, then review the top k cases per day. Here, k is fixed by operational resources — for example, an analyst team might investigate $k = 100$ cardholders per day. The **Card Precision@k** metric measures the fraction of cardholders in the top k who are truly compromised.

Unlike traditional metrics, this approach:

- Accounts for multiple transactions from the same cardholder.
- Focuses directly on the operational "hit rate".
- Enables simulation of multi-day investigation strategies.

Step 1: Daily Precision Calculation (`card_precision_top_k_day`). This function computes the daily Card Precision@k value:

1. **Group** transactions by `CUSTOMER_ID`.
2. **Select per card:**
   - The *maximum predicted fraud probability* from that day's transactions.
   - The *maximum observed fraud label* — 1 if any transaction is fraudulent.
3. **Rank** cardholders by predicted probability in descending order.
4. **Select** the top k cards.
5. **Compute precision:**

$$\text{Card Precision@k(day)} = \frac{\text{\# of fraudulent cards in top } k}{k}.$$

6. **Return** both the precision value and the IDs of detected compromised cards.

Step 2: Multi-Day Simulation (`card_precision_top_k`). Since fraud detection operates continuously over time, this function simulates investigation over multiple days, considering the removal of detected cards:

1. **Loop through days** (sorted by `TX_TIME_DAYS`) in the test set.
2. **Remove detected cards** from future days if `remove_detected_compromised_cards = True`, simulating the blocking of fraudulent cards upon detection.
3. **Call** `card_precision_top_k_day` to compute that day's precision.
4. **Track:**
   - The number of *new* compromised cards in that day's top k.
   - The daily Card Precision@k metric.
5. **Aggregate:**

$$\overline{\text{Card Precision@k}} = \frac{1}{D}\sum_{d=1}^{D} \text{Card Precision@k}(d),$$

   where D is the number of operational days in the evaluation period.

Integration in `performance_assessment`. The evaluation pipeline calls both transaction-level and card-level metrics, producing results such as:

- ROC AUC score.
- Average Precision score.
- Mean Card Precision@k for each k in the analyst's capacity list (e.g., 50, 100, 200).

Interpretation. Card Precision@k directly quantifies the ability to *prioritise* cards that are truly compromised in the first k inspections of each day. A model with strong ROC AUC but poor Card Precision@k may rank fraudulent transactions well overall but fail to bring the correct cardholders into the investigator's immediate action list.

Summary. This chapter has detailed both the standard transaction-level metrics (AUC ROC, Average Precision, Precision, Recall) and the custom card-level measure *Card Precision@k*, including the rationale for its introduction and the mechanics of

its daily and multi-day computation. Together, these metrics provide a comprehensive framework for assessing fraud detection models in both statistical and operational terms.

The following chapter (Chapter 5) presents the computed metrics for all models under study, comparing their performance and interpreting the results in the context of practical fraud investigation constraints.

Chapter 5

Results and Discussion

5.1 Overview

This chapter presents and discusses the empirical results obtained by applying the evaluation metrics described in Chapter **??**. All models were assessed on both transaction-level metrics (AUC ROC, Average Precision) and the operationally targeted card-level metric (*Card Precision@k*).

5.2 Test Set Performance

Table 5.1: Model performance on the test set for $k = 100$. Best results in each column are highlighted in bold.

Model	**AUC ROC**	**Average Precision**	**Card Precision@100**
Logistic Regression	**0.899**	0.739	0.110
Decision Tree (max depth = 2)	0.846	0.619	0.100
Decision Tree (unlimited)	0.863	0.506	0.101
Random Forest	0.897	**0.777**	**0.110**
Gradient Boosting	0.866	0.736	0.107

On the test set:

- **AUC ROC:** Logistic Regression achieved the highest score (0.899), closely followed by Random Forest (0.897), indicating strong ranking ability for both linear and ensemble approaches.

- **Average Precision:** Random Forest provided the strongest precision–recall balance (0.777), suggesting superior handling of the extreme class imbalance.
- **Card Precision@100:** Both Logistic Regression and Random Forest tied for the top value (0.110), meaning roughly 11% of the top 100 flagged cards per day were truly compromised.

5.3 Training Set Performance

Table 5.2: Model performance on the training set for $k = 100$. Best results in each column are highlighted in bold.

Model	AUC ROC	Average Precision	Card Precision@100
Logistic Regression	0.901	0.739	0.218
Decision Tree (max depth = 2)	0.859	0.651	0.207
Decision Tree (unlimited)	1.000	1.000	**0.253**
Random Forest	**1.000**	**1.000**	**0.253**
Gradient Boosting	0.872	0.749	0.213

The training set results clearly show overfitting in the unlimited-depth Decision Tree and Random Forest models, which achieve perfect AUC and Average Precision. Their high *Card Precision@100* on the training set (0.253) drops substantially on the test set (0.110), confirming that these scores do not generalise.

5.4 Execution Times

Table 5.3: Training and prediction execution times (in seconds) for each model.

Model	Training Time (s)	Prediction Time (s)
Logistic Regression	2.215	0.066
Decision Tree (max depth = 2)	2.740	0.082
Decision Tree (unlimited)	37.031	0.125
Random Forest	275.276	1.459
Gradient Boosting	363.415	0.966

The execution time analysis highlights a trade-off between complexity and efficiency:

- **Fastest models:** Logistic Regression and the shallow Decision Tree have minimal training and prediction overhead.

- **Slowest training:** Gradient Boosting required over 360 seconds to train.
- **Slowest prediction:** Random Forest was the most computationally expensive at inference (1.46 s), which could limit scalability in real-time detection environments.

5.5 Discussion

The main takeaways in these performance results are as follows:

- **All prediction models learned useful fraud patterns.** This is evident from the AUC ROC on the test set, which is well above the random baseline of 0.5 for all classifiers (ranging from 0.846 to 0.899), and from the Average Precision values, which are much higher than the baseline fraud rate of 0.7% (ranging from 0.506 to 0.777). This confirms that the classifiers are not predicting at random and capture meaningful signals in the data.
- **Random Forest is the best overall performer.** On the test set, it achieved an AUC ROC of 0.897 and the highest Average Precision of 0.777, indicating strong discriminative ability and good handling of class imbalance.
- **Logistic Regression is a strong, simple baseline.** With an AUC ROC of 0.899 and an Average Precision of 0.739, it performs surprisingly well, demonstrating that when fraud patterns are reasonably separable, a linear classifier can be very competitive.
- **Gradient Boosting is competitive but slightly behind.** It obtained an AUC ROC of 0.866 and an Average Precision of 0.736, placing it just below Random Forest and Logistic Regression in both metrics.
- **Overfitting in the unlimited-depth Decision Tree.** In training, it achieved perfect scores (AUC ROC = 1.000, Average Precision = 1.000), but on the test set, performance dropped sharply to 0.863 (AUC ROC) and 0.506 (Average Precision). This indicates memorization of training data and poor generalization. In contrast, the shallow Decision Tree (max depth = 2) showed more conservative training results (AUC ROC = 0.859, Average Precision = 0.651) but relatively stable performance on the test set (AUC ROC = 0.846, Average Precision = 0.619), highlighting the trade-off between model complexity and generalization.
- **Card Precision@100 reveals similar operational ranking across models.** On the test set, values for Card Precision@100 are similar across classifiers

(around 10%–11%), meaning that all models identify roughly the same proportion of true fraud cases among the top 100 suspicious cards per day. However, on the training set, complex models such as Random Forest and the unlimited-depth tree reach 25.3%, which again reflects overfitting.

- **Execution times show clear efficiency differences.** Logistic Regression and the shallow Decision Tree train in only 2–3 seconds and predict in milliseconds, making them highly efficient for deployment. Random Forest (275 s) and Gradient Boosting (363 s) require significantly more training time due to their ensemble nature, though their prediction times remain under two seconds. This computational cost should be weighed against the performance benefits when choosing a production model.

In summary, Random Forest offers the best balance of performance and generalization on this dataset, Logistic Regression remains a strong and extremely fast baseline, and overly complex models such as unlimited-depth Decision Trees should be avoided due to their pronounced overfitting.

These findings will guide the recommendations in Chapter 7 regarding optimal deployment strategies for fraud detection, balancing detection accuracy with resource constraints.

Chapter 6

Implementation of Findings, Limitations, and Future Work

6.1 Implementation of Findings

The analysis in Chapter 5 demonstrates that fraud detection models can vary widely in predictive performance, computational requirements, and tendency to overfit. In operational terms, these findings can be implemented as follows:

- **Adopt Random Forest for high-accuracy batch processing.** Random Forest's superior Average Precision (0.777) and competitive AUC ROC (0.897) make it well suited for offline scoring of large transaction datasets, where detection completeness is prioritised over minimal latency.
- **Deploy Logistic Regression for real-time scoring.** Its extremely low prediction latency (0.066 s) and strong performance (AUC ROC 0.899) mean it can be integrated into live transaction streams for instant fraud flagging.
- **Avoid overly complex unregularised tree models.** The unlimited-depth Decision Tree's severe overfitting indicates that in the absence of regularisation or ensemble averaging, deep single trees do not generalise well.
- **Incorporate Card Precision@k as a monitoring KPI.** This metric has direct operational meaning for analyst workloads, ensuring model outputs align with daily investigation capacity.

These implementation pathways assume secure integration into the card issuing bank's detection systems, with regular model retraining to capture evolving fraud patterns.

6.2 Limitations of the Study

While the results are encouraging, the study has several limitations:

- **Synthetic data environment.** The dataset, though realistic, was simulated. Real-world transaction data may include fraud patterns or biases not covered here.
- **Metric alignment.** Although Card Precision@k was tailored for operational utility, its k value and performance targets may differ across institutions.
- **Static training/test split.** The evaluation did not account for temporal drift. In production systems, fraud behaviour and transaction characteristics change over time.
- **Limited feature engineering.** The models operated on aggregated features from the simulator. More sophisticated temporal or behavioural predictors might improve performance.

6.3 Future Work

Potential future developments include:

- **Validation on real banking data.** Testing with historical transaction logs from a financial institution, subject to strict privacy compliance.
- **Inclusion of temporal validation.** Adopting a rolling-window evaluation to capture the impact of fraud concept drift.
- **Advanced model architectures.** Exploring deep learning models (e.g., LSTM networks for sequential data) and hybrid methods combining statistical and graph-based features.
- **Adaptive thresholding.** Using dynamic decision thresholds that adapt to changing fraud prevalence and investigation capacity.

Chapter 7

Conclusions

7.1 Summary of Contributions

This work presented a complete framework for training, evaluating, and operationalising fraud detection models in a credit card transaction environment. The main contributions are:

- Development of a realistic synthetic transaction data simulator, incorporating customer and terminal profiles, transaction generation, and fraud injection.
- Implementation of baseline and advanced classifiers, including Logistic Regression, Decision Trees, Random Forest, and Gradient Boosting.
- Proposal and evaluation of a novel operational metric, *Card Precision@k*, designed to align detection performance with daily analyst investigation constraints.
- Empirical comparison of models across statistical and operational metrics, highlighting trade-offs between complexity, accuracy, and computational efficiency.
- Practical recommendations for deployment, balancing Random Forest's accuracy with Logistic Regression's speed.

7.2 Final Reflections

Fraud detection remains a moving target, where adversaries continuously adapt to circumvent detection systems. While advanced ensemble methods can extract subtle patterns, their computational and maintenance costs must be justified for each operational context. Metrics tailored to analyst capacity—such as Card Precision@k—help ensure

that model improvements translate into operational value, rather than only statistical gains.

The findings suggest that bridging the gap between machine learning research and financial crime operations requires not only accurate models, but also meaningful evaluation, constant monitoring, and adaptability in the face of emerging fraud strategies.

In closing, this study demonstrates that effective fraud detection is not achieved by blindly maximising model accuracy, but by aligning modelling choices with real-world constraints, stakeholder needs, and the evolving nature of financial fraud.

References

1. Grover, P., Xu, J., Tittelfitz, J., Cheng, A., Li, Z., Zablocki, J., Liu, J., & Zhou, H. (2023). *Fraud Dataset Benchmark and Applications*. Preprint, under review. https://github.com/amazon-research/fraud-dataset-benchmark

2. Hernandez Aros, L., Bustamante Molano, L. X., Gutierrez-Portela, F., Moreno Hernandez, J. J., & Rodriguez Barrero, M. S. (2024). Financial fraud detection through the application of machine learning techniques: a literature review. *Humanities & Social Sciences Communications, 11*(130). https://doi.org/10.1057/s41599-024-03606-0

3. Fraud Detection Benchmark GitHub Repository. (2023). Amazon Research Team. https://github.com/amazon-research/fraud-dataset-benchmark

4. Reproducible Machhine Learning for Credit Card Fraud detection. (2025). https://fraud-detection-handbook.github.io/fraud-detection-handbook/

5. Fraud detection in Python. (2025). https://trenton3983.github.io/posts/fraud-detection-python/